Os princípios **SOLID, DRY, KISS e YAGNI** são fundamentos da engenharia de software para criar códigos limpos, fáceis de manter e escaláveis. [1, 2, 3]

---

1. SOLID (Cinco Princípios da Programação Orientada a Objetos)

Criado por Robert C. Martin (Uncle Bob), o **SOLID** é um acrônimo para cinco regras de design de código: [1, 2]

- **S — Single Responsibility Principle (Princípio da Responsabilidade Única):** Uma classe ou módulo deve ter **um, e apenas um, motivo para mudar**. Ela deve executar uma única tarefa ou papel no sistema. [1]
- **O — Open/Closed Principle (Princípio Aberto/Fechado):** As entidades de software devem estar **abertas para extensão, mas fechadas para modificação**. Você deve conseguir adicionar novas funcionalidades sem reescrever o código existente. [1]
- **L — Liskov Substitution Principle (Princípio da Substituição de Liskov):** Objetos de uma subclasse devem poder **substituir objetos da superclasse** sem quebrar o funcionamento ou a lógica do programa. [1]
- **I — Interface Segregation Principle (Princípio da Segregação de Interface):** É melhor ter **várias interfaces específicas** do que uma única interface genérica gigante. Nenhuma classe deve ser forçada a implementar métodos que não utiliza.
- **D — Dependency Inversion Principle (Princípio da Inversão de Dependência):** Dependa de **abstrações, e não de implementações concretas**. Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de interfaces. [1]

---

2. DRY — *Don't Repeat Yourself* (Não Se Repita)

- **O que é:** Defende que **toda peça de conhecimento deve ter uma representação única, ambígua e autoritativa** dentro do sistema.
- **Na prática:** Se você copiou e colou o mesmo bloco de código em três lugares diferentes (o famoso *Ctrl+C, Ctrl+V*), você violou o DRY. A solução é extrair essa lógica para uma **função, método ou componente** reutilizável.
- **Benefício:** Quando a regra de negócio mudar, você altera o código em **um único lugar**, evitando bugs por inconsistência. [1, 2, 3, 4]

---

3. KISS — *Keep It Simple, Stupid* (Mantenha Simples)

- **O que é:** Afirma que a **maioria dos sistemas funciona melhor se mantida simples do que se tornar complexa**.

- **Na prática:** Evite "engenharia excessiva" (*overengineering*). Escreva códigos diretos, legíveis e fáceis de entender por qualquer outro desenvolvedor da equipe.
- **Benefício:** Reduz o tempo de manutenção, facilita novos testes e diminui a curva de aprendizado para quem entra no projeto. [1, 2, 3, 4]

---

4. YAGNI — *You Aren't Gonna Need It* (Você Não Vai Precisar Disso)

- **O que é:** Orienta que você **nunca deve adicionar uma funcionalidade** baseando-se apenas na premissa de que *"talvez a gente vá precisar disso no futuro"*. [1, 2, 3]
- **Na prática:** Implemente estritamente o que foi pedido para o **momento atual**. Código preditivo não solicitado vira lixo técnico, consome tempo e dificulta mudanças reais. [1, 2, 3]
- **Benefício:** Foco no valor real entregue no presente e código muito mais enxuto.